

What This Code Builds

A line-by-line, plain-language walkthrough of main.py

1. What Are We Actually Building?

This script builds a small **AI agent that can do math using external tools**, instead of just guessing the answer from its training data. Think of it as three pieces working together:

- **The Brain (LLM)**: A language model (running on Groq's servers) that can understand a question in plain English and decide what to do about it.
- **The Toolbox (MCP Server)**: A separate small Python program (math_server.py) running on your own machine, which knows how to actually add and multiply numbers correctly and exactly — no guessing.
- **The Bridge (MCP Client + LangChain Agent)**: This main.py file. It starts the math server, connects to it, hands its tools to the LLM, and lets the LLM decide when to call those tools to solve a problem.

So the flow is: **You ask a math question → the agent (LLM) reads it → realizes it needs the 'add' or 'multiply' tool → calls that tool on the math server → gets the exact answer back → replies to you in a sentence.** This is the core pattern behind "AI agents with tools" and is exactly what MCP (Model Context Protocol) was designed for: letting an LLM safely call external programs instead of only using its own memory.

Why not just let the LLM do the math itself?

Language models are good at language, not arithmetic — they can get large or unusual calculations wrong because they're predicting text, not truly computing. By giving the model a real calculator tool (the MCP math server) and letting it call that tool, you get language understanding + guaranteed-correct computation, instead of relying on the model to "guess" the right number.

2. How the Pieces Connect (Integration Overview)

- ① **StdioServerParameters** tells Python exactly how to launch the math server as a separate background process (which command, which script file).
- ② **stdio_client** actually starts that process and opens two communication pipes to it: one to send messages (write), one to receive replies (read). "stdio" means the two programs talk to each other through standard input/output, like two people passing notes back and forth.
- ③ **ClientSession** wraps those two pipes into a proper MCP conversation — it knows the MCP protocol's rules for asking "what tools do you have?" and "please run this tool."
- ④ **load_mcp_tools** asks the math server what tools it offers (add, multiply) and converts them into a format LangChain / the LLM can understand and call.
- ⑤ **create_agent** combines the LLM (ChatGroq) with those tools into one "agent" object that can reason and decide when to use a tool.
- ⑥ **agent.ainvoke(...)** is where you actually ask your question. The agent figures out the steps, calls the right tool(s) on the math server, and returns a final answer.

In short: the top part of the file is all *setup and wiring* (get the model ready, get the tool server ready, connect them). The bottom part (`main()`) is where the actual conversation and tool-calling happens.

3. Line-by-Line Explanation

Setting Up — The Imports

```
import asyncio
```

Brings in Python's built-in library for running "async" code — code that can pause while waiting for something slow (like a network call or another program replying) without freezing the whole script. We need this because talking to the math server and the LLM both involve waiting for responses.

```
from dotenv import load_dotenv
load_dotenv()
```

This reads a hidden `.env` file in your project folder and loads any secret values in it (like your Groq API key) into the environment, so the rest of the code can use them without you typing your API key directly into the script. This keeps secrets out of your code.

```
from langchain_core.agents import HumanMessage
```

Imports the **HumanMessage** class, which is just a standard way of wrapping "a message from the user" so LangChain and the LLM know it came from a human, not the AI itself. *(Note: the correct import path for `HumanMessage` is actually `langchain_core.messages`, not `langchain_core.agents` — this line works right now but is technically importing from the wrong place and could break in a future version.)*

```
from langchain_groq import ChatGroq
```

Imports the connector class that lets LangChain talk to Groq's servers, which host fast, free-tier large language models (like Llama or GPT-OSS). This is "the brain" of the agent.

```
from langchain_mcp_adapters.tools import load_mcp_tools
```

Imports a helper function whose only job is: "connect to an MCP server, ask what tools it has, and translate those tools into a format LangChain agents understand." This is the glue between the MCP world and the

LangChain world.

```
from mcp import ClientSession, StdioServerParameters
```

Imports two pieces from the core MCP (Model Context Protocol) library: **ClientSession** (manages a live conversation with an MCP server) and **StdioServerParameters** (describes how to launch that server as a local process).

```
from mcp.client.stdio import stdio_client
```

Imports the function that actually starts the math server program and opens the two-way pipe to talk to it over "stdio" (standard input/output) — the simplest way for two programs on the same computer to exchange messages.

```
from langchain.agents import create_agent
```

Imports the modern LangChain function for building an agent: give it a model and a list of tools, and it returns a ready-to-use agent that can reason step by step and call tools when needed. (This replaced the older, now-deprecated `create_react_agent` from `LangGraph`.)

Setting Up — The Model and the Server Location

```
llm = ChatGroq(model="openai/gpt-oss-120b")
```

Creates the actual language model connection. `model="openai/gpt-oss-120b"` tells Groq exactly which model to use for answering. This `llm` object is what will later read your question and decide what to do.

```
stdio_server_params = StdioServerParameters(
    command="python",
    args=["C:/.../servers/math_server.py"],
)
```

This doesn't start anything yet — it just writes down the *instructions* for how to start the math server later: "run the command `python` with this script file as the argument." It's like writing a recipe before you actually cook it.

The Main Program — Where Everything Actually Runs

```
async def main():
```

Defines the main function as "async," meaning it's allowed to pause and wait for slow steps (starting a process, waiting for a reply) without blocking the whole program.

```
    async with stdio_client(stdio_server_params) as (read, write):
```

This line actually **launches the math server** as a background process, using the instructions from earlier. It opens two channels: `read` (to receive messages from the server) and `write` (to send messages to it). The `async with` also guarantees the process is properly shut down afterward, even if something goes wrong.

```
        async with ClientSession(read_stream=read, write_stream=write) as session:
```

Wraps those raw `read/write` channels into a proper **MCP session** — an object that knows how to speak the MCP "language" (asking for tool lists, sending tool calls, etc.), instead of you having to write that low-level protocol logic yourself.

```
            await session.initialize()
            print("session initialized")
```

Performs the initial handshake with the math server (like saying "hello, are you ready?") and waits for confirmation before doing anything else. The `print` statement is just a visual confirmation for you, the developer, that the connection succeeded.

```
            tools = await load_mcp_tools(session)
```

Asks the math server, through the session, "what tools do you offer?" — and gets back a ready-to-use list of tools (here: `add` and `multiply`) already converted into the format LangChain agents expect.

```
            agent = create_agent(llm, tools)
```

Combines the brain (`llm`) with the toolbox (`tools`) into one agent object. This agent now knows both how to think in language and which real tools it's allowed to call.

```
            result = await agent.ainvoke(
                {"messages": [HumanMessage(content="What is 52 + 2 * 4?")]}
            )
```

This is the actual question being asked. It's wrapped as a `HumanMessage` inside a "messages" list (the standard format LangChain agents expect for a conversation). `ainvoke` runs the agent: the LLM reads the

question, decides it needs the multiply and add tools, calls them on the math server, and builds a final answer. `await` means the code pauses here until that whole process finishes.

```
print(result["messages"][-1].content)
```

The agent's full conversation (including any tool calls and tool results behind the scenes) is stored in `result["messages"]`. `[-1]` grabs the *last* message in that list — which is the agent's final, human-readable answer — and prints just its text content to the screen.

Program Entry Point

```
if __name__ == "__main__":  
    asyncio.run(main())
```

A standard Python pattern meaning "only run this if the file is executed directly (not imported into another file)." `asyncio.run(main())` is what actually kicks off the whole async process defined above, from start to finish.

4. Quick Reference — What Each Import Is For

Import	Plain-Language Purpose
asyncio	Lets the program pause on slow steps without freezing
load_dotenv	Loads your secret API key from the .env file
HumanMessage	Marks a message as coming from the user
ChatGroq	Connects to the LLM (the "brain") hosted by Groq
load_mcp_tools	Fetches and converts the math server's tools for LangChain
ClientSession / StdioServerParameters	Manage the connection & startup instructions for the tool server
stdio_client	Actually launches the math server and opens communication pipes
create_agent	Builds the agent by combining the LLM with its tools

One thing worth flagging: the HumanMessage import comes from langchain_core.agents in this file, but its real home is langchain_core.messages. It happens to still work, but fixing the import path would make the code more future-proof.